

Article

Mitigating the Risks of Malware Attacks with Deep Learning Techniques

Abdullah M. Alnajim ^{1,*}, Shabana Habib ¹, Muhammad Islam ², Rana Albelaihi ³
and Abdulatif Alabdulatif ⁴

¹ Department of Information Technology, College of Computer, Qassim University, Buraydah 51452, Saudi Arabia; s.habibullah@qu.edu.sa

² Department of Electrical Engineering, Unaizah College of Engineering, Qassim University, Buraydah 51452, Saudi Arabia

³ Department of Computer Science, College of Engineering and Information Technology, Onaizah Colleges, Onaizah 56447, Saudi Arabia; ralbelaihi@oc.edu.sa

⁴ Department of Computer Science, College of Computer, Qassim University, Buraydah 51452, Saudi Arabia; ab.alabdulatif@qu.edu.sa

* Correspondence: najim@qu.edu.sa

Abstract: Malware has become increasingly prevalent in recent years, endangering people, businesses, and digital assets worldwide. Despite the numerous techniques and methodologies proposed for detecting and neutralizing malicious agents, modern automated malware creation methods continue to produce malware that can evade modern detection techniques. This has increased the need for advanced and accurate malware classification and detection techniques. This paper offers a unique method for classifying malware, using images that use dual attention and convolutional neural networks. Our proposed model has demonstrated exceptional performance in malware classification, achieving the remarkable accuracy of 98.14% on the Maling benchmark dataset. To further validate its effectiveness, we also evaluated the model's performance on the big 2015 dataset, where it achieved an even higher accuracy rate of 98.95%, surpassing previous state-of-the-art solutions. Several metrics, including the precision, recall, specificity, and F1 score were used to evaluate accuracy, showing how well our model performed. Additionally, we used class-balancing strategies to increase the accuracy of our model. The results obtained from our experiments indicate that our suggested model is of great interest, and can be applied as a trustworthy method for image-based malware detection, even when compared to more complex solutions. Overall, our research highlights the potential of deep learning frameworks to enhance cyber security measures, and mitigate the risks associated with malware attacks.

Keywords: attention module; cyber security; deep learning; malware classification



Citation: Alnajim, A.M.; Habib, S.; Islam, M.; Albelaihi, R.; Alabdulatif, A. Mitigating the Risks of Malware Attacks with Deep Learning Techniques. *Electronics* **2023**, *12*, 3166. <https://doi.org/10.3390/electronics12143166>

Academic Editor: Andrei Kelarev

Received: 31 May 2023

Revised: 12 July 2023

Accepted: 17 July 2023

Published: 21 July 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Malware, often known as malicious software, is a category of software that is specifically created to cause harm to, or to damage, computer systems. Its primary objective is to harm computer systems deliberately, by attacking, penetrating, or obtaining unauthorized access to sensitive digital assets, which may cause unwanted consequences or damage to the system [1]. In 2020, on average, 360,000 new malware files were discovered daily, increasing by 5.2% [2]. The proliferation of malware has been facilitated by the availability and utilization of sophisticated and automated malware creation tools, such as Zeus SpyEye, as well as denial-of-service (DoS) attacks [3]. Emerging blended attacks pose more significant threats, combining multiple assault types for an increased impact. Hacking, spoofing, phishing, and spyware incidents are surging, with a noticeable rise in deceptive phishing attacks. The vulnerable Internet of Things (IoT) faces cyberattacks and viruses, resulting in data breaches and manipulation, and profoundly affecting society.

This information highlights the severe threat that malware and cyberattacks pose to our interconnected digital world [4–6]. Efficient cybersecurity is vital to safeguarding IoT users from malware threats, on connected devices and smart appliances. Malware detection techniques have evolved from labor-intensive manual labeling to advanced hybrid systems [7]. The application of association rule mining and various other techniques to anti-malware software has increased the creation of new malware. Malware classification involves dynamic analysis (observing malware behavior during its runtime) and static analysis (examining the properties of malware binaries without their execution) [8]. Dynamic analysis is a widely used technique for malware analysis, but it has drawbacks, such as the analysis being time-consuming, and the potential damage caused by the malware. New methods [9–13] have been developed to overcome these limitations. Traditional malware detection techniques rely on feature engineering and expert knowledge, but struggle to keep up with rapid malware development. Signature-based methods are also becoming insufficient against new automatic malware generation techniques. To enhance detection capabilities, new techniques are required [14,15]. Machine learning models have become more popular recently in various fields, and the latest platforms have adopted image processing methods and deep learning or machine learning techniques to categorize malware. The most popular method for identifying malware based on its features is to merge machine learning algorithms and artificial neural networks (ANNs) into more complicated designs, such as ensemble learning [8–15]. Several studies have used neural networks (NNs) and support vector machines (SVMs) as popular choices for malware classification and adversarial attacks [16]. Furthermore, feature selection and classifier hyperparameter tuning can be accomplished using nature-inspired and metaheuristic optimization techniques, such as a genetic algorithm (GA). These techniques have successfully classified malware [17,18]. Deep learning and image processing algorithms are used for malware classification without intensive feature engineering. Converting malware binaries into images helps identify specific malware types, as different types within a family share identical image structures. In recent years, researchers have used more complex neural network architectures to improve the malware classification accuracy by combining convolutional neural networks (CNNs) and recurrent neural networks (RNNs), such as long short-term memory (LSTM), with other machine learning models or hybrid models [19–21]. This study employs advanced data augmentation techniques to enhance the performance of the malware detection model. Traditional signature and heuristic approaches to malware identification do not offer an adequate level of detection for novel and previously unidentified kinds of malware. This will determine whether or not ML techniques can be applied to resolve this issue. Advanced deep learning techniques, in conjunction with transfer learning [22] strategies without the requirement for in-depth security knowledge, are employed to increase the robustness and accuracy of malware detection.

This study aims to develop a state-of-the-art solution, using cutting-edge methods for malware recognition. The contributions of this study are outlined below:

Contributions:

- **Improved Accuracy with Spatial Attention:** The proposed model incorporates spatial attention mechanisms, improving the malware classification accuracy. By selectively focusing on the relevant input regions, the model achieves a higher precision in feature extraction, and an enhanced classification performance.
- **Novel Combination of Attention Mechanisms:** The study introduces a unique combination of spatial attention and convolutional neural networks, shedding light on the effectiveness of dual-attention mechanisms in malware classification. This approach provides valuable insights into optimizing attention mechanisms for computer vision tasks.
- **Enhancement of MobileNetv1 Performance:** By integrating spatial attention and channel attention mechanisms, the proposed model effectively enhances the performance of the MobileNetv1 model. This advancement contributes to improving the accuracy of computer vision models in various applications, beyond malware classification.

- **Practical Application in Malware Detection:** The experimental results demonstrate the proposed model's practicality as a trustworthy method for image-based malware detection. Its high accuracy, comparable to more complex solutions, showcases its potential for real-world deployment in cybersecurity systems.

The paper is organized as follows. Firstly, the related work in the field of malware detection techniques, including feature engineering and attention mechanisms, is discussed. This section provides an overview of the existing approaches, and highlights the need for novel techniques to address the limitations of traditional methods. Next, the proposed methodology is presented, outlining the integration of spatial attention and convolutional neural networks for malware classification. The architecture and design choices are explained in detail, emphasizing the advantages of the proposed model. Subsequently, the experimental setup and results are presented, demonstrating the model's performance on the Malimg benchmark dataset. Evaluation metrics such as the precision, recall, specificity, and F1 score are used to assess the accuracy and robustness of the model. Additionally, an ablation study is conducted, to analyze the impact of different components of the proposed model on its performance. Finally, the paper concludes with a summary of its contributions. It highlights the potential of deep learning frameworks and attention mechanisms to enhance cybersecurity measures, in the face of evolving malware threats.

Related Work:

In a study by Rezende et al. [23], ResNet-50 was utilized to create a neural network architecture that used transfer learning. The uniform Glorot approach was utilized for weight initialization, and the input to the network was made up of RGB images, with dimensions of 224×224 . The model was trained for 750 epochs, using Adam optimization, with the final accuracy of 98.62% obtained using 10-fold cross-validation. With the addition of GIST features and the k-nearest neighbors algorithm (kNN), with k set to 4, the accuracy was 97.48%. Once the bottleneck features were applied, the accuracy reached 98.0%. Khan et al. [24] performed a thorough analysis, including the data preparation pipeline and top model, to assess the efficiency of transfer learning in malware classification, using ResNet and GoogleNet. The analysis of ResNet's 18, 34, 50, 101, and 152 performance yielded accuracy ratings of 83%, 86.51%, 86.62%, 85.94%, and 87.98%. On the other hand, GoogleNet provided an accuracy of 84%. Vasan et al. [25] utilized an ensemble model consisting of VGG16 and ResNet-50, both of which were tuned; their model was adjusted on 50 epochs, the CNN model was trained on 100 to 200 epochs, and the final result was an accuracy of 99.50%. In addition, ref. [26] 90% of the features in the dataset were reduced using PCA, before being fed into a one-versus-all multiclass SVM. Yosinski et al. [27] presented a model with 15 classes developed using a dataset of 7087 samples. We used a variety of feature extraction algorithms, and our most outstanding accuracy was 97.47%. Nataraj et al. [25] used machine learning algorithms such as kNN, and feature extraction techniques such as GIST descriptors in their study, achieving an accuracy of 97%. They calculated the bigram distributions, and employed static feature categorization in their technique. The key problem of this method is that if an attacker is aware of the functions being utilized, they can evade detection by implementing countermeasures. Akarsh et al. [28] examined the integration of hybrid LSTMs or CNNs with SVMs and other hybrid SVM architectures, as well as different deep learning models. In their investigation, the hybrid GRU-SVM and MLP-SVM models outperformed the CNN-SVM model in accuracy, scoring 84.92% and 80.46%, respectively. Akarsh et al. [29] proposed a hybrid CNN-LSTM model with a novel method of picture manipulation. They suggested a two-layer CNN, with an LSTM layer with 70 memory blocks, an F.C.N. layer with 25 units, and a categorical cross-entropy softmax layer as its foundation. The model's ultimate accuracies ranged from 96.64% to 96.68% on various data distributions. In a separate study, Akarsh et al. [30] utilized two layers of 1D CNN and LSTM for feature extraction, along with 70 LSTM memory blocks, and 0.1% dropouts, into their model. They also used a cost-sensitive approach. Their model's 95.5% accuracy score was its highest. In their study [31], Sudhakar and Kumar enhanced the ResNet-50 malware classification model, by substituting a fully connected

dense layer for the final layer of the pre-trained ImageNet model. The SoftMax layer then used the output from this dense layer to categorize the virus. Ember, an approach proposed by Vinayakumar et al. [32], used domain-specific knowledge, various features extracted from the analyzed PE (portable performance) files, and format-independent features, such as a raw byte histogram.

The authors of Xiao et al. [33] proposed MalFCS, a methodology for classifying malware that represented malware binaries as entropy graphs derived from structural entropy. Based on these entropy plots, deep convolutional neural networks were also employed to find shared patterns among the malware family. The malware was finally classified using SVM and the extracted features. In their research, Cui et al. [34] suggested employing the “Bat Algorithm” for dynamic image resampling, to address dataset imbalance. In their convolutional neural network (CNN) model, they reached 94.5% accuracy by combining this approach with data augmentation techniques. Cui et al. [35] conducted a study in which they presented a data-smoothing approach using the NSGA-III genetic algorithm. The results showed that, without data smoothing, their approach yielded an accuracy of 92.1%. However, when a single objective algorithm was used, the accuracy increased to 96.1%. Using the multi-objective algorithm, the highest accuracy of 97.1% was achieved. Jain et al. [36] utilized extreme learning machines (ELMs) coupled with convolutional neural networks (CNNs), and presented an ensemble model. With a single CNN layer, their model achieved 96.30% accuracy. Similarly, with two CNN layers, the accuracy was 95.7%. In their study, Naeem et al. [37] employed a hybrid visualization technique based on deep learning and the Internet of Things (IoT). They successfully built models with the great accuracy of up to 98.47% and 98.79%, by incorporating various aspect ratios. It should be emphasized, nonetheless, that this accuracy depended on the image’s dynamic features [16]. In their research, Venkatraman et al. [38] showed a self-learning system with a hybrid architecture. By fusing a CNN with BiLSTM and BiGRU, they created hybrid models, which they trained using cost-sensitive and cost-insensitive techniques. Their models, which used a variety of parameters and settings, displayed levels of accuracy between 94.48% and 96.3%. Wu et al. [39] suggested a convolutional neural network (CNN)-based architecture, with byte-class, gradient, Hilbert, entropy, and hybrid image transform (HIT), using GIST and CNN-based models, as well as many image transformations performed to the input images. They discovered that applying the GIST transformation to their grayscale photos produced an accuracy rate of 94.27%. However, their most potent model used the HIT technique combined with a CNN. El-Shafai et al. [40] presented a method for classifying various malwares that combined transfer learning with pre-trained CNN models (AlexNet, DenseNet-201, DarkNet-53, ResNet-50, Inception-V3, VGG16, MobileNet-V2, and Places365-GoogleNet). Their research showed that VGG16 demonstrated the best malware recognition performance. Moussas and Andreatos [41] created a two-layer artificial neural network (ANN) that detected malware using both file and picture data. The malware was categorized using file features at the first level of the ANN, and the perplexing malware families were categorized using malware image features at the second level of the ANN. Roseline et al. [42] employed deep learning methods to recognize and categorize malware. This method created discriminative representations from the data themselves, rather than depending on manually created feature descriptors. The suggested approach beats deep neural networks at malware detection, thanks to deep file stacking and a simple model.

Verma et al. [43] suggested an ensemble-learning-based malware classification technique integrating second-order statistical texture features, based on a first-order matrix and grey co-occurrence matrix (GLCM). On the Malimg dataset, they used an extreme learning machine (ELM) classifier with a kernel-based performance of 94.25% accuracy. Çayır et al. [44] employed the CapsNet file model for malware categorization in their study. CapsNet uses a straightforward approach to architecture engineering, instead of sophisticated CNN architectures and domain-specific feature-engineering methodologies. Additionally, CapsNet is easy to train from scratch, and does not require transfer learning. The authors Wozniak et al. [45], for Android malware detection, proposed an RNN-LSTM

classifier with the NAdam optimization technique. On two benchmark datasets, the performance of their suggested strategy was assessed, and the findings indicated the high accuracy of 99%.

Nisa et al. [46] used segmentation-based fractal texture analysis (SFTA) to extract features from images containing malware code, and combine them with features from deep neural networks already readying AlexNet and Inception-v3. Different classifiers, including SVM, kNN, and decision trees (DTs) were employed to categorize the features retrieved from malware images. A study by Hemalath et al. [47] used a weighted class-balanced loss function, along with the DenseNet model. This method successfully addressed the issue of unbalanced data, which significantly increased the classification accuracy of malware photos. MJ Awan et al. [2] suggested utilizing VGG19 pre-trained deep neural networks as an attention-based model. To extract significant features from VGG19, the attention module was used. As a result, this method performed better, resulting in the high accuracy of 97.62%. S Depuru et al. [48] proposed a neural network model for classifying malicious attacks, by evaluating different combinations of malware representation methods and convolutional neural network (CNN) models. The selected model achieved the high accuracy of 96%. S Yaseen et al. [49] utilized deep learning models, specifically convolutional neural networks (CNNs), to classify malware families. By transforming malware binaries into grayscale images, and employing CNNs, the proposed method achieved the impressive accuracy of 97.4%. Mallik et al. [50] utilized convolutional recurrence with grayscale images, BiLSTM layers, data augmentation, and convolutional neural networks, achieving the remarkable accuracy of 98.36% in malware classification. K Gupta et al. [51] utilized an artificial neural network architecture to precisely classify malware variants, effectively tackling the challenges posed by obfuscation and compression techniques. The experimental results demonstrated an accuracy of 90.80%.

2. Methodology

This section discusses the critical insights gained from our approach to using dual-attention CNNs for malware detection, which resulted in improved detection rates compared to previous methods. Our approach consists of (1) transfer learning for features extractions, and (2) the design of a dual-attention CNN for image detection. The framework of the proposed method is illustrated in Figure 1. The framework consists of data pre-processing, and transfer learning-based feature extraction. In the data pre-processing stage, image data are acquired from the malware binary files, and subjected to data augmentation techniques to enhance their diversity. The framework's second stage involves leveraging the MobileNet [52] model for feature extraction. Consequently, to improve the feature representation, dual-attention mechanisms are utilized.

2.1. Data Pre-Processing

Pre-processing is an important step in machine learning that enriches and increases the data quality, for better training. As stated, garbage in, garbage out CNN models learn well if the data are enhanced, and information is improved while training; as a result, the model performs well on unseen data. We performed pre-processing steps, as discussed briefly below, to improve the model's generalization and performance.

2.1.1. Image Dimensions

During the training and testing stages, the image dimensions feeding the model must be the same. In this study, we maintained the images dimensions at 224×224 , and used a higher-quality image to extract more features from the images. Additionally, we normalized the image pixel values between (0,1) to allow the model to learn smoothly.

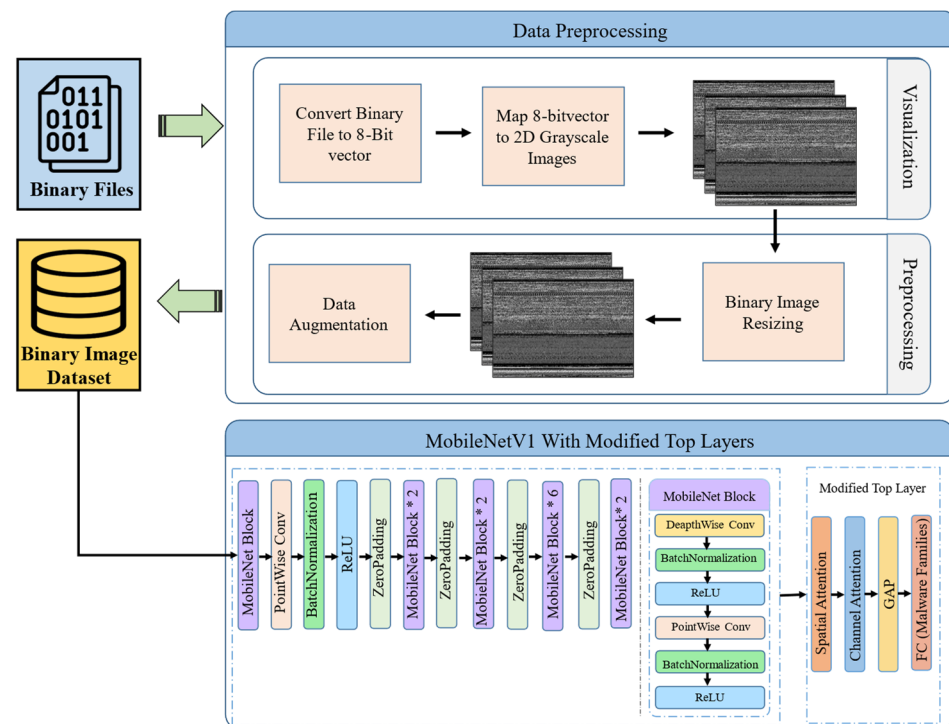


Figure 1. Framework of the proposed model.

2.1.2. Data Augmentation

Deep learning models are data-hungry; they perform well if enough data are available for training. Additionally, data improve model generalization; on the other hand, data augmentation is a technique to generate training examples [53]. Furthermore, it works sufficiently to prevent overfitting. Therefore, we used data augmentation to improve the model performance, and increase the model generalization. More precisely, transformation was applied to the images, while varying the original image's data transformation of the spatial location of images, with the assumption of keeping the essential features unchanged. The types of augmentation applied are given below.

Rotation images: This transformation rotates images, and creates new variations of the same images, increasing the dataset's variety and model generalization. The model becomes exposed to a broader range of variations, including rotations. This transformation reduces bias in the dataset.

Flipping image: This transformation transforms horizontal and vertical flips in images, which helps the model to be more robust to variations that may occur in unseen data.

Contrast Enhancement: This adjusts the effects of contrast variations in an image by varying the light conditions. This helps to improve the visual quality of images through an increasing contrast between the pixels. This enhances the clarity of an image, making it easier to identify essential features. This transformation helps in increasing the model generalization, and focusing on the critical features.

Zoom Images: This transformation refers to changing the scale of the image by either zooming in (making it larger) or zooming out (making it smaller). The transformation increases the variety in the dataset, which helps in the model generalization to unseen data.

As well as these transformations, we applied other transformations, such as scaling, shifting, and color transformation, to the data. Conclusively, the data augmentation technique generated new samples of data, while retaining the essential features of the images. Figure 2 provides a visual representation of the output resulting from the data augmentation process, illustrating the augmented images with the applied transformations.

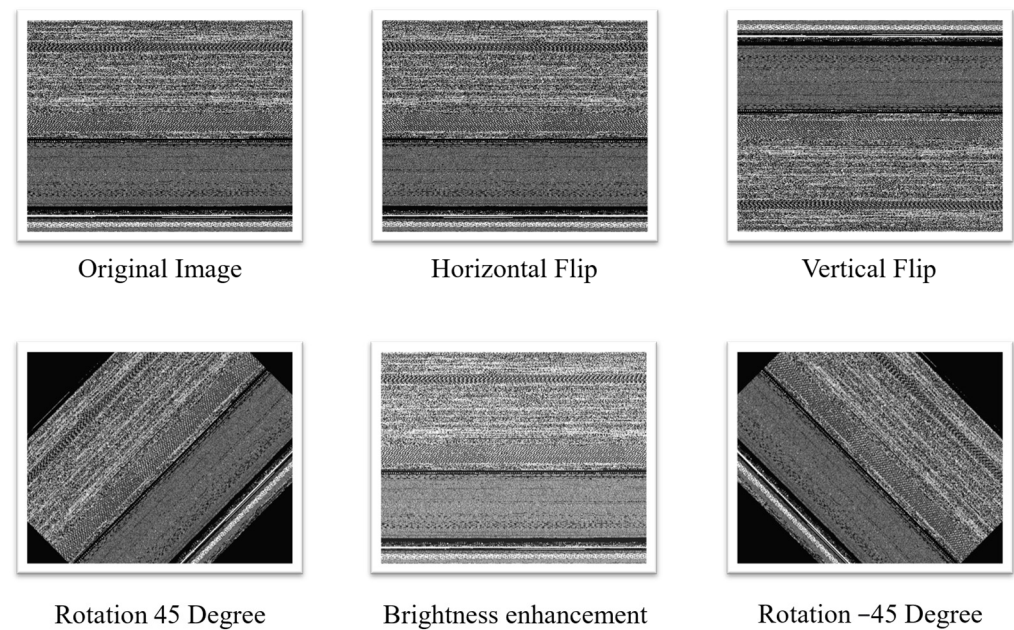


Figure 2. Data augmentation samples.

2.2. Transfer Learning

Deep learning performs well when enough data and computation are available [54–57]. In many cases, there are limitations in either the specific data availability, or the computational resources [58]. Hence, data-specific tasks are prohibitive because of these limitations. However, transfer learning offers a better solution to these problems [59]. Transfer learning is a technique wherein a pre-trained convolutional neural network is trained on a generic and big dataset, and the learned weights are utilized for the new task, even if there are fewer data points and a lack of enough computing [54]. By using transfer learning, we simplified the training process, and accelerated the time taken by the neural network. Figure 3 demonstrates an example of fine-tuning in transfer learning, as documented in the literature [60]. In this study, we used the pre-trained model, i.e., MobileNetV1 [52], which trained on a big dataset, namely ImageNet, and tweaked the model for malware classification.

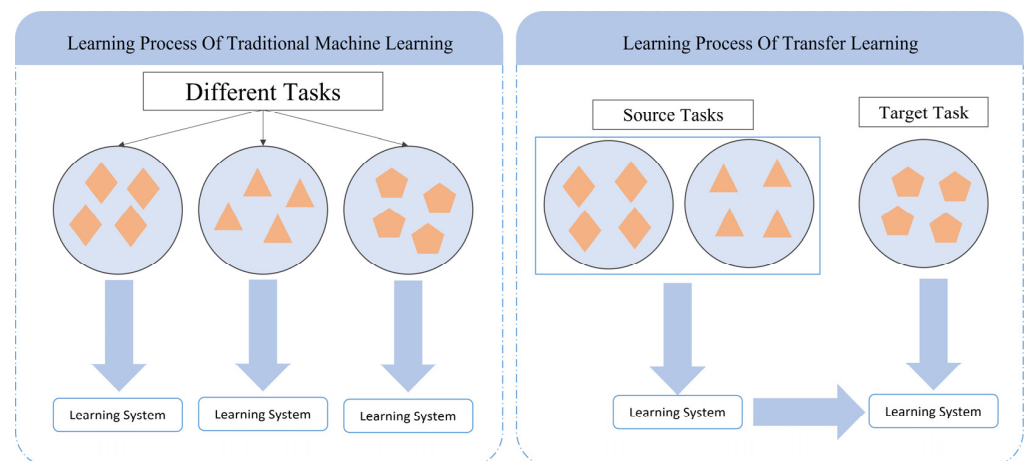


Figure 3. Transfer learning and traditional learning.

2.3. MobileNet

MobileNetV1 is a highly efficient deep learning model designed explicitly for mobile and embedded devices [52]. It achieves this efficiency by using depth-separable convolutions, which consist of a depthwise convolution and a pointwise convolution.

Equation (1) states that in traditional convolutions; which have the kernel parameters $DK \times DK \times M \times N$, where $DK \times DK$ is the size of the convolution kernel, M is the number of input channels, and N is the number of output channels; depth-separable convolutions have the much smaller kernel parameters $DK \times DK \times 1 \times M$ for depth convolution, and $1 \times 1 \times M \times N$ for point convolution.

$$\frac{D_K * D_K * M * N * D_F * D_F}{D_K * D_K * M * D_F * D_F + M * N * D_F * D_F} = \frac{1}{N} + \frac{1}{D_K^2} \quad (1)$$

This simplified architecture significantly reduces the computational cost of the model. Standard convolutions have a computational cost of $DK \times DK \times M \times N \times DF \times DF$, while depth-separable convolutions have the much lower cost of $DK \times DK \times M \times DF \times DF + M \times N \times DF \times DF$, especially when using a 3×3 convolution kernel. This reduction in the computational complexity allows MobileNetV1 to achieve a high accuracy while operating efficiently on mobile devices, utilizing the CPU and GPU to accelerate image recognition, without sacrificing accuracy. MobileNetV1 is designed to decompose standard convolutions into depth-separable ones, significantly reducing the number of parameters and the computational complexity. In this study, the pre-trained convolutional base of MobileNet was maintained, and global average pooling was used after the last convolutional layer. The features are first passed through spatial attention, followed by channel attention. Afterward, the last fully connected layer uses a softmax classifier. Figure 4 shows the MobileNet architectural model proposed in this study. More details on the MobileNet architecture can be found in the literature [61].

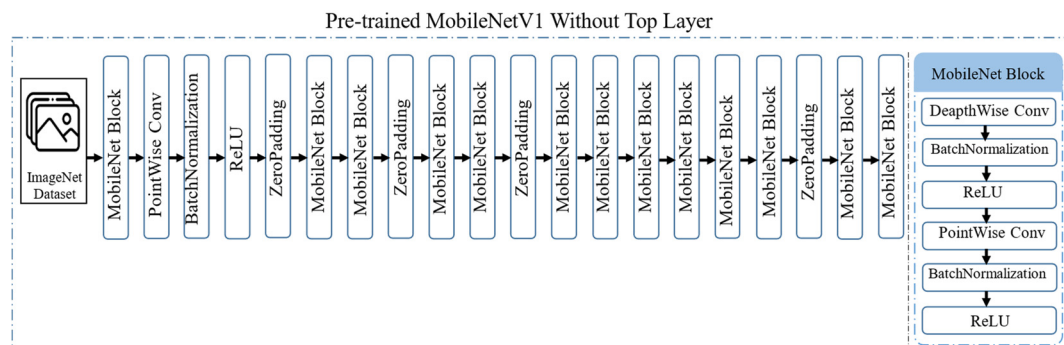


Figure 4. MobileNetV1 architecture without the top layer.

2.4. Spatial Attention

Spatial attention is a mechanism that narrows a neural network's focus onto specific regions or areas of an input image or feature map [62,63]. The neural network learns the weight for each pixel in each channel of the feature map, which generates a spatial attention map, where each weight shows the importance of the pixel [64]. The network can filter out irrelevant essentials, and focus on necessary features, which can help model accurate predictions [64].

If we let X (belongs to $R(H, W, C)$) be the output feature obtained from the last convolutional layer of the pre-trained MobileNet, then the dimension of the feature map obtained from Mobile Net is $7 \times 7 \times 1024$, which is passed to the spatial attention (SA) module. The SA first calculates the feature map average, which calculates the mean of the spatial dimension. Afterward, the features are passed across the spatial dimensions of the map, where the maximums values are filtered out. To produce attention, the feature maps and original features are concatenated and passed through a tiny convolutional layer, with one output map, with the kernel size of 3×3 , using the activation function of sigmoid, and with the output of the layer reduced to the ratio $r = 16$ [65], to reduce computational costs. The SA module is illustrated in Figure 5.

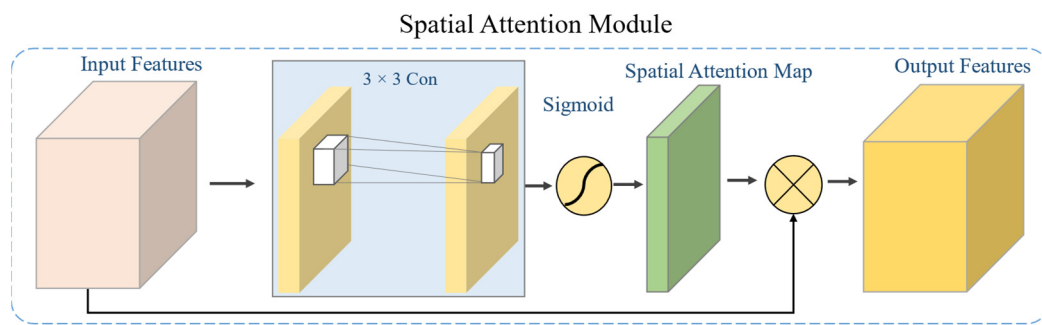


Figure 5. Spatial attention module.

2.5. Channel Attention

Each channel in a feature map is a feature detector for a specific feature [66]. Channel attention is mainly used for learning the inter-channel chain of feature maps, which means that neural networks focus on the most essential feature maps [63,66]. In addition, it helps to predict the image class [64]. To determine or filter out the essential features from spatial attention, we employed channel attention (CA) after spatial attention in the following way: we used average-pooling, followed by the max-pooling of each feature in the feature map, to add up the information throughout the map. The outputs of both the average-pooling and max-pooling are concatenated together, and are further passed through a small convolutional neural network (CNN) with one output map, using a 3×3 kernel with the dimension reduction ratio $r = 16$ [65], activated by the sigmoid [67] activation function. The original feature maps comprise, together with channel attention maps, the output of a small CNN.

The output of the channel attention is first flattened to a 1-d vector, and then passed to the classifier layer activated by the softmax [67] activation function. The CA module is illustrated in Figure 6.

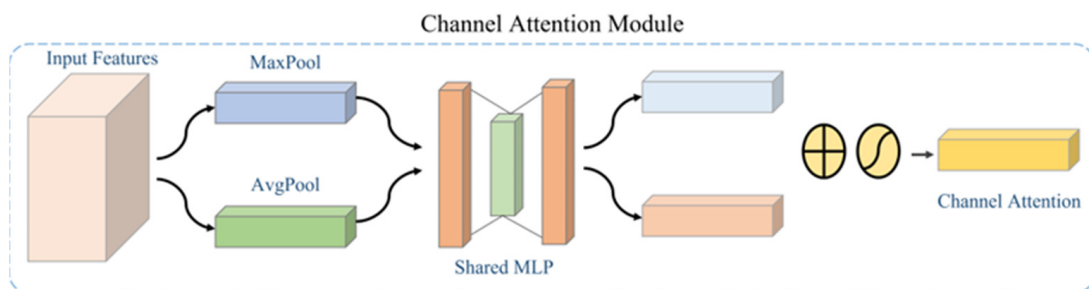


Figure 6. Channel attention module.

3. Experimental Setup and Result

In this section, we present the evaluation of the effectiveness of our proposed method. We performed experiments based on the malware dataset [25]. Additionally, we used the famous neural network framework TensorFlow [68], and the higher-level API Keras [69] to create and train dual-attention-based CNNs. We performed the experiments on the hardware with the following specification: PC with an Intel Core i7-8590 CPU (3.3 GHz), Nvidia GeForce G.T.X. 750Ti GPU (2 GB), and 8 GB RAM. Using TensorFlow and Keras, we implemented the pre-trained MobileNet, using an Adam optimizer, with a learning rate of 0.001 and batch size 4. The model was trained for 40 epochs.

3.1. Maling Dataset

The Maling dataset, called the malware image dataset, was developed by Nataraj et al. [25]. This dataset consists of 9389 grayscale images belonging to 25 distinct malware families. It is frequently used as a standard benchmark for evaluating malware detection

techniques in academic research, including those specifically intended for implementation in IoT settings [37,70]. To construct the images, malware binaries were transformed into 8-bit vectors, which were then translated into grayscale images. The intensity of each pixel in the image corresponds to the value of the vector. The dataset includes a variety of prominent malware families and their associated variants. Figure 7 provides a visual representation of the process that generates a malware image from a binary file.

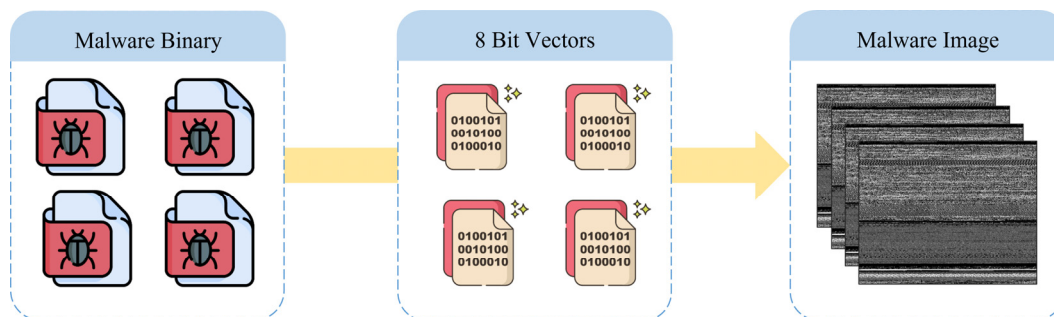


Figure 7. The process of generating a malware image from a binary file.

The dataset comprises 9389 malware images from 25 different classes, with Allaple. A and Allaple. L the most frequent malware types. The dataset is highly imbalanced, with significant variations in the number of samples in each class. The visualization of the dataset's distribution can be effectively demonstrated through a bar chart, as depicted in Figure 8.

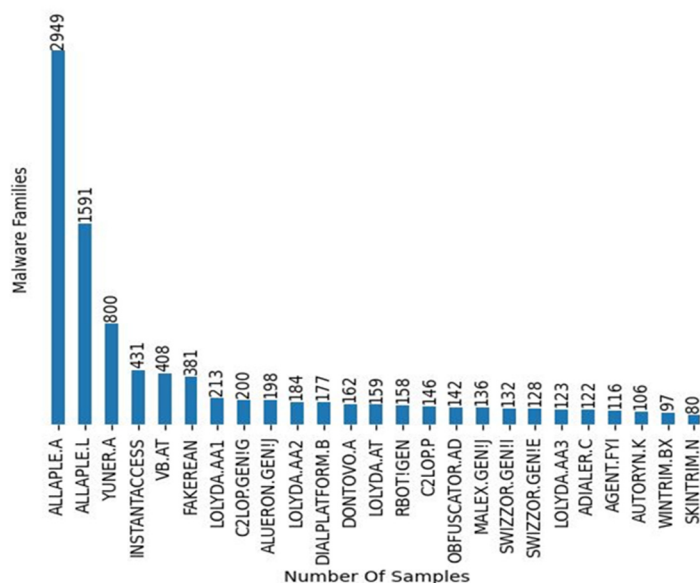


Figure 8. Maling dataset distribution.

3.2. Big 2015 Dataset

In this paper, we utilized the malware dataset known as BIG 2015, obtained from the Microsoft Malware Classification Challenge on Kaggle. The dataset consists of two main parts: a training dataset and a testing dataset. The training dataset comprises 10,868 malware programs, classified into nine distinct categories. The dataset consists of two files for each malware sample: a .bytes file and a .asm file. The .bytes file contains the raw hexadecimal code, while the .asm file contains the disassembled assembly code of the malware. We used the .bytes file to generate images for the experiment, as illustrated in Figure 7. This allowed us to effectively represent the malware samples, and extract meaningful features for our analysis. Figure 8 showcases a selection of image samples

from the dataset, providing visual representations of the malware. These images offer a glimpse into the diverse characteristics and patterns present in the dataset, enabling a better understanding of the visual features utilized in our analysis.

Figure 9 presents a collection of malware images representing different classes within the dataset. These images correspond to the following malware families: (a) Adialer.C, (b) Autorun.K, (c) Obfuscator.ACY, (d) Ramnit, (e) Dinwold, and (f) Regrun. Each image visually captures the distinct characteristics and features of the respective malwares, facilitating their identification, and enabling our researchers to analyze their behaviors and potential threats.

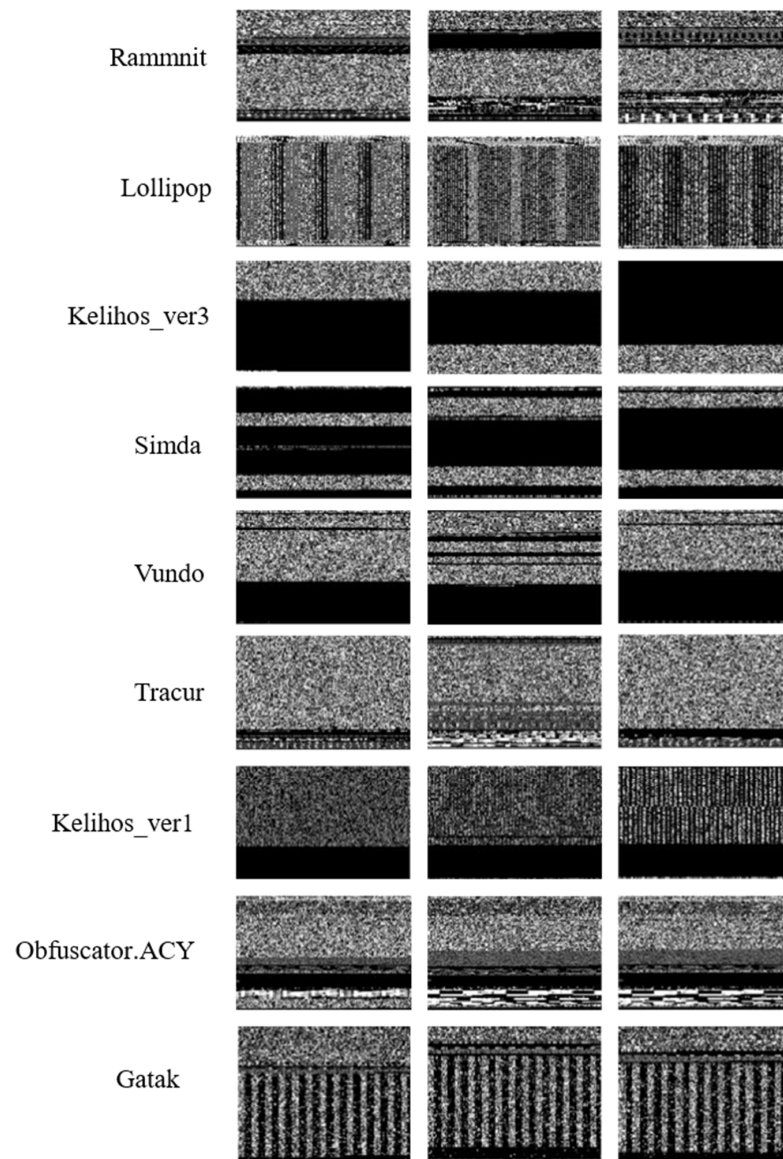


Figure 9. Big 2015 dataset samples.

The big 2015 dataset comprises 10,868 malware images from nine different classes, with Lollipop and Kelihos ver3 being the most frequent malware types. The dataset exhibits a significant class imbalance, with variations in the number of samples in each class. A bar chart is utilized to visually represent the dataset's distribution, as depicted in Figure 10. This bar chart provides valuable insights into the dataset's composition, and the prevalence of different malware classes, as observed in the accompanying image.

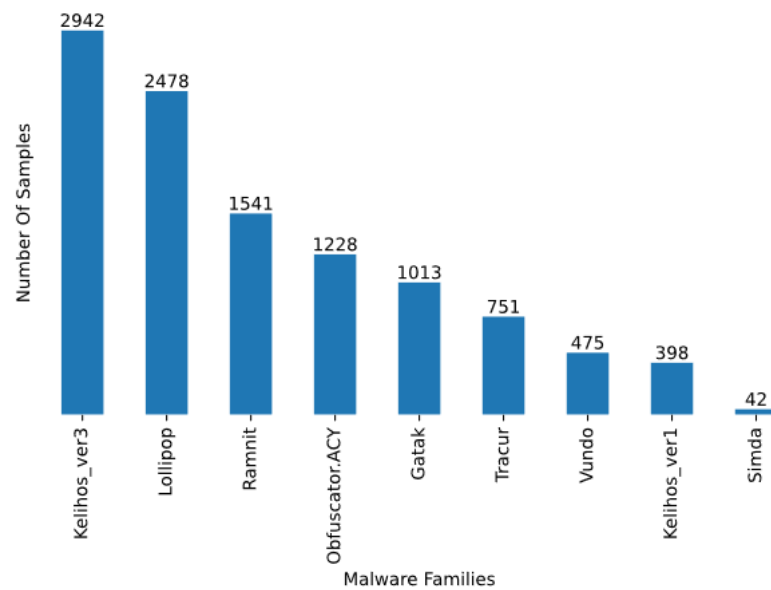


Figure 10. Big 2015 dataset distribution.

3.3. Evaluation of Results

This section provides an overview of the evaluation metrics used in our experiments. Mainly, the research community used the accuracy, F1 score, precision, and recall to evaluate the model's performance in the test dataset testing process, regarding the classification model, object recognition in machine learning, deep learning, and information retrieval [71].

Accuracy: the ratio of correctly classified instances by the total number of instances in the dataset, as given in Equation (2).

Precision: the proportion of correctly classified positive (true positive) instances by the total number of instances classified as positive and false instances, calculated using Equation (3).

Recall or Sensitivity: the proportion of true positives by the total number of true positive and false negative instances, calculated using Equation (4).

F1 score: the harmonic mean of the precision and recall, calculated using Equation (5).

$$\text{Acc} = \frac{\text{Correctly Predicted Samples}}{\text{Total Samples in the set}} \times 100 \quad (2)$$

$$\text{Precision} = \frac{\sum_{i=1}^l TP_i}{\sum_{i=1}^l (TP_i + FP_i)} \quad (3)$$

$$\text{Recall} = \frac{\sum_{i=1}^l TP_i}{\sum_{i=1}^l (TP_i + FN_i)} \quad (4)$$

$$\text{F1 score} = \frac{(\text{Precision} \times \text{recall})}{(\text{Precision} + \text{recall})} \times 2 \quad (5)$$

The TP (true positive) indicates positive data classified as positive; FP (false positive) is a negative class classified as positive. TN (true negative) is the positive class classified as negative, and FN (false negative) is the negative class classified as negative.

3.4. Experimental Results

We carried out experiments to assess the proposed model's efficiency and performance: (1) a mobile net with other deep learning (DL) model with no attention module (AM), (2) a mobile net with other DL with AM, and (3) a comparison with state-of-the-art (SOTA) models.

3.4.1. Experimental Results on the Maling Dataset

The study conducted in this research primarily revolves around the analysis of the Maling dataset, which consists of a comprehensive collection of malicious images. The evaluation and analysis carried out in this study specifically focus on the Maling dataset, to thoroughly assess the performance and effectiveness of the proposed model in the context of malicious image classification. The results and conclusions drawn from this study primarily apply to the Maling dataset, allowing for a detailed understanding of the model's capabilities in handling this specific dataset.

Comparison of the Proposed Model with Other Deep Learning Models with No Attention Module

To verify the performance of our proposed model without the attention module with other deep learning models, we compared the results of the proposed model with other deep learning models. The performance of the different classification models and the proposed model (written in bold) is illustrated in Table 1. From the table, it can be seen that the proposed model outperforms the other deep learning models.

Table 1. Comparison between the proposed model and other deep learning models.

Model	Accuracy	Loss
DenseNet121	0.9521	6.07869
EfficientNetB0	0.9675	1.74945
VGG16	0.9651	2.54061
ResNet-101	0.9662	1.23805
MobileNetV1	0.9732	0.03596

Figure 11 shows the training and validation accuracy (left) and loss (right) of the proposed model, but with no attention module employed. We trained the model for 40 epochs. The training accuracy of the proposed model increased smoothly at the beginning. However, there was no learning curve on the validation sets. Similarly, the training loss was significantly lower, but the validation loss showed poor learning. As a result, the model could not learn well, and lacked generalization. There is still a gap for improvement in the model.

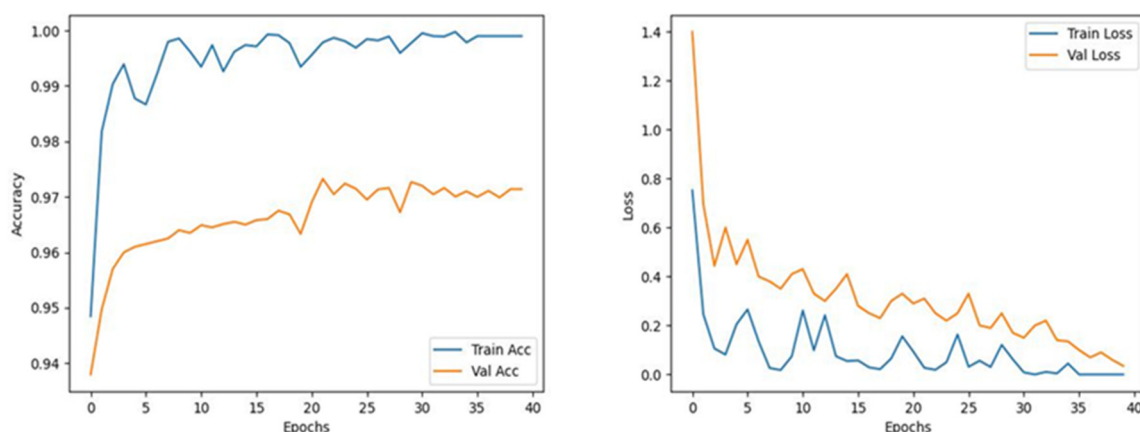


Figure 11. The training and validation accuracy and loss of the proposed model, without an attention module.

Figure 12 shows the confusion matrix for the proposed model without using the attention module, and shows the incorrect prediction value and the accurate value for each class.

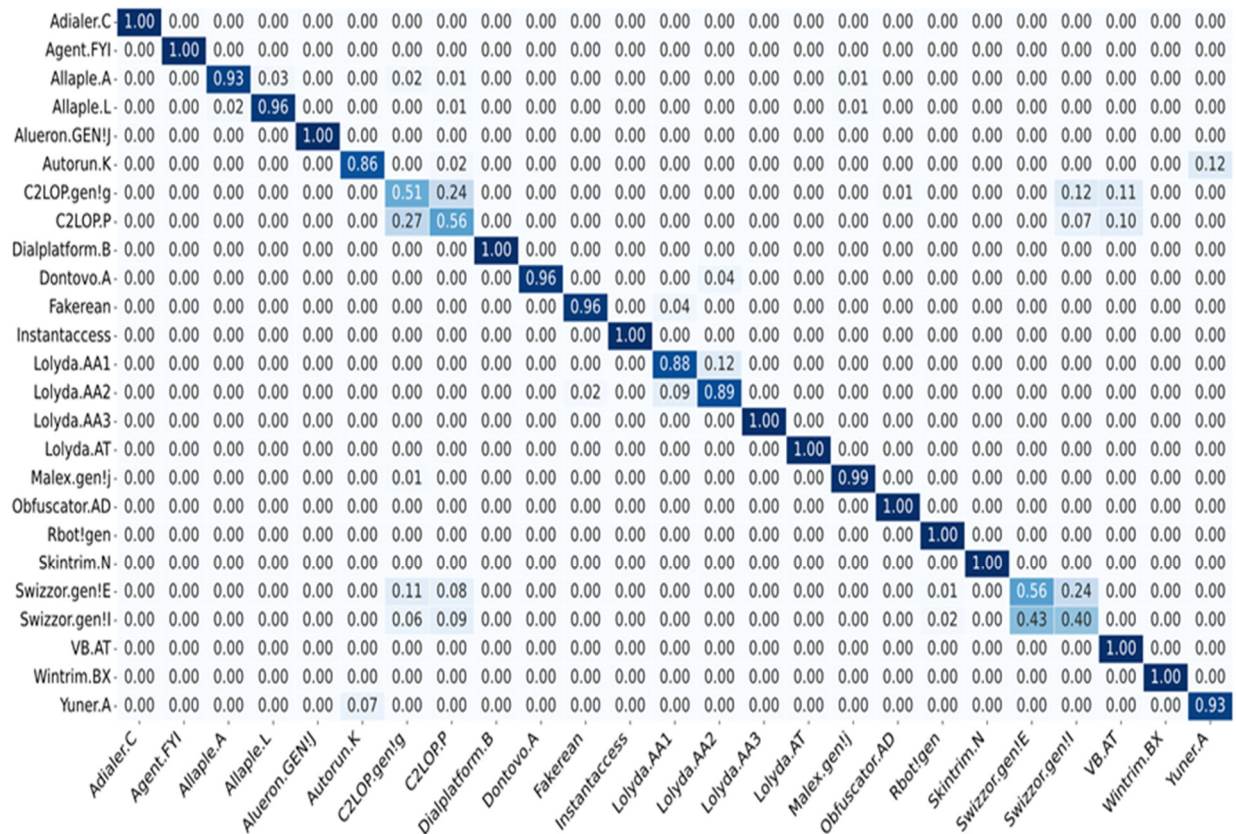


Figure 12. Confusion matrix of our model without attention module.

Table 2 shows the classification report, where we can see the obtained precision, recall, F1 score, and the overall accuracy achieved at 97.32% on the test data.

Table 2. Experimental results of our proposed model with no attention module.

Maling Family	Precision	Recall	F1 Score	Support
ALLAPLE.E	0.97	0.96	0.97	125
ALLAPLE.L	0.98	0.97	0.97	100
YUNER.A	0.96	0.95	0.95	75
INSTANTACCESS	0.96	0.97	0.96	75
VB.AT	0.96	0.96	0.96	60
FAKEREAN	0.97	0.97	0.97	25
LOLYDA.AA1	0.97	0.96	0.96	25
C2LOP.GEN!G	0.95	0.96	0.96	25
ALUERON.GEN!J	0.96	0.96	0.96	25
LOLYDA.AA2	0.97	0.97	0.97	25
DIALPLATFORM.B	0.96	0.97	0.97	25
DONTOVO.A	0.96	0.95	0.95	25
LOLYDA.AT	0.95	0.95	0.95	25
RBOT!GEN	0.97	0.97	0.97	25
C2LOP.P	0.94	0.94	0.94	25
OBFUSCATOR.AD	0.97	0.96	0.96	25
MALEX.GEN!J	0.96	0.96	0.96	25
SWIZZOR.GEN!I	0.96	0.97	0.96	25
SWIZZOR.GEN!E	0.97	0.98	0.97	25
LOLYDA.AA3	0.97	0.96	0.96	25
ADIALER.C	0.97	0.96	0.96	25
AGENT.FYI	0.96	0.97	0.97	25
AUTORYN.K	0.95	0.95	0.95	25
WINTRIM.BX	0.96	0.96	0.96	25
SKINTRIM.N	0.97	0.97	0.96	25
Accuracy			0.973	935
Macro avg	0.96	0.96	0.96	935
Weighted avg	0.97	0.97	0.97	935

Comparison of the Proposed Model with Other Deep Learning Models with an Attention Module

Deep learning models work well if the feature extractor extracts more essential features. To extract more information from the features, we employed an attention module with the proposed model and other deep learning for assessment. Table 3 illustrates the results of the proposed model with an attention module, with other deep learning models. The proposed model outperformed all the other deep learning models. Figure 13 shows the accuracy and loss of the proposed model with an attention module. At the beginning of the training and validation, the accuracy (left) increased very smoothly, and an accuracy of 98% was achieved on 40 epochs. Additionally, the proposed model's training and validation loss (right) decreased significantly, and reached 0.002. Overall, the attention module empirically showed that it learns the best, and achieves maximum accuracy.

Table 3. Comparison of the proposed model with an attention module.

Model	Accuracy	Loss
DenseNet121	0.930481	0.319
EfficientNetB0	0.973262	0.313
VGG16	0.971123	1.051
ResNet-101	0.976471	0.457
(Proposed) MobileNetV1	0.981403	0.002

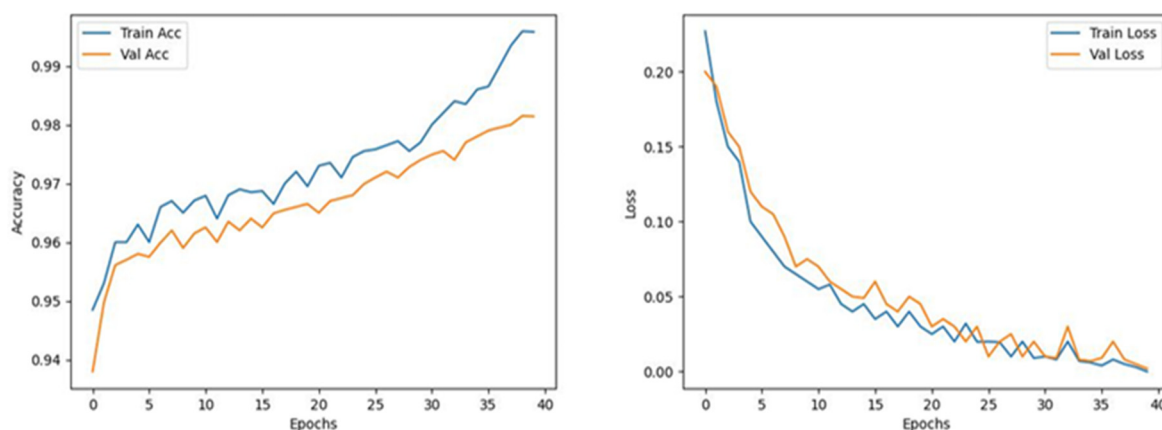


Figure 13. Training and validation accuracy and loss of the proposed model with an attention module.

The confusion matrix of the proposed model with an attention module, displayed in Figure 14, represents the actual labels along the y -axis, and the predicted labels along the x -axis.

Table 4 shows the classification report of the proposed method with an attention module; we can see the precision, recall, and F1 score. The confusion matrix of the proposed model using the attention module stage is shown in Figure 11; the matrix shows the correct and incorrect values of each class. The test accuracy was 98.14%. Notably, the SC classification performance improved significantly with the attention module.

3.4.2. Experimental Results on the Big 2015 Dataset

The following results represent the performance evaluation conducted on the Big 2015 dataset. This dataset, obtained from Microsoft, is a large-scale collection of malware samples. The analysis and assessment focus on the proposed model's effectiveness in classifying the malware instances within the Big 2015 dataset. The findings and conclusions presented in this study shed light on the model's performance, and its potential for detecting and categorizing malware accurately in real-world scenario

Comparison of the Proposed Model with Other Deep Learning Models with No Attention Module

The performance of our proposed model, which does not include an attention module, was compared with other deep learning models, to validate its effectiveness. The results of this comparison are presented in Table 5, where the performance of various classification models is evaluated alongside the proposed model (highlighted in bold). Upon examining the table, it becomes evident that the proposed model exhibits a superior performance compared to the other deep learning models.

Table 5. Comparison of the proposed model without an attention module.

Model	Accuracy	Loss
DenseNet121	0.9601	6.0775
EfficientNetB0	0.97754	1.7494
VGG16	0.9620	2.5406
ResNet-101	0.9750	1.1080
MobileNetV1	0.9881	0.1171

In Figure 15, we present the training and validation accuracy (left) and loss (right) of the proposed model, which does not include an attention module, over 40 epochs. During the initial training phase, the model's training accuracy steadily increased. However, the validation accuracy did not exhibit a noticeable learning curve, indicating a lack of progress in generalizing to unseen data. Furthermore, while the training loss consistently decreased, the validation loss remained relatively high, suggesting poor learning and a limited generalization ability beyond the training data. These observations indicate that the proposed model, without the attention module, struggled to learn the underlying patterns and complexities within the dataset effectively. Therefore, there is a clear need for improvement in the model's performance. Further research and experimentation are required, to enhance its learning capabilities, address the lack of generalization, and narrow the gap between the training and validation performance.

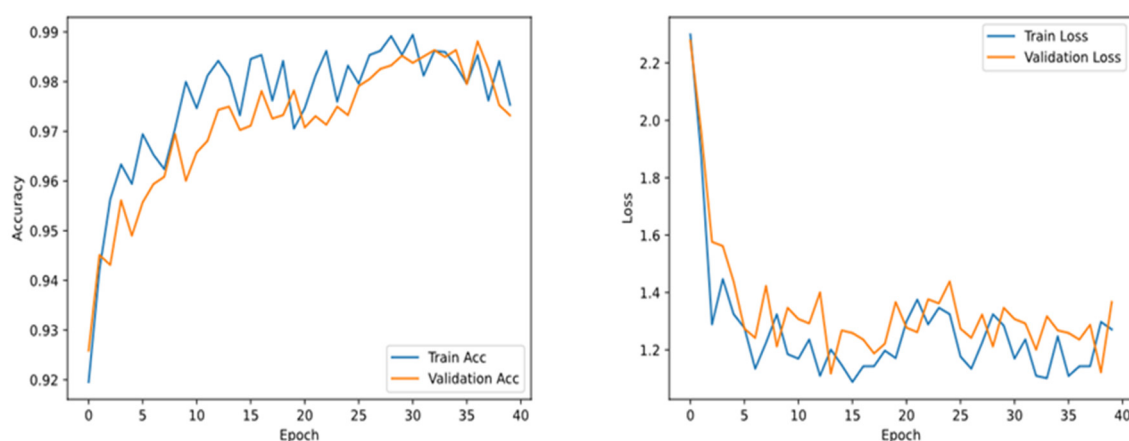


Figure 15. Training and validation accuracy and loss of the proposed model with no attention module.

Figure 16 represents the confusion matrix for the proposed model without the attention module. The confusion matrix provides valuable insights into the model's performance, by displaying the counts of correct and incorrect predictions for each class. By examining the confusion matrix, we can identify the classes where the model frequently misclassifies instances, leading to higher numbers in the false prediction values. On the other hand, the accurate prediction values indicate the classes where the model demonstrates a strong performance, and correctly assigns labels.



Figure 16. Confusion matrix of the proposed model with no attention module.

Table 6 shows the classification report of the proposed model without an attention module, where we can see the obtained precision, recall, and F1 score, and the overall accuracy achieved at 98.81% on the test data.

Table 6. Classification report of the proposed model with no attention module.

Big 2015 Family	Precision	Recall	F1 Score	Support
Ramnit	0.99	0.99	0.99	100
Lollipop	0.98	0.96	0.97	125
Kelihos_ver3	0.99	0.99	0.99	133
Vundo	0.98	0.99	0.99	25
Simda	0.99	0.98	0.98	20
Tracur	0.99	0.99	0.99	80
Kelihos_ver1	0.98	0.99	0.99	45
Obfuscator.ACY	0.99	0.98	0.99	93
Gatak	0.99	0.99	0.99	90
Accuracy			0.99	711
Macro avg	0.99	0.99	0.99	711
Weighted avg	0.99	0.99	0.99	711

Comparison of the Proposed Model with Other Deep Learning Models with an Attention Module

In deep learning models, the effectiveness greatly relies on feature extraction, which aims to capture essential information. We integrated an attention module into the proposed model to enhance the feature extraction capability, and compared its performance with other deep learning models. Table 7 presents this assessment's results, showcasing the proposed model's performance with the attention module, alongside other deep learning models. Remarkably, the proposed model outperformed all the other models, highlighting its superior ability to extract meaningful features, and achieve better classification accuracy. This finding reinforces the significance of incorporating an attention module into deep learning models, to enhance their feature extraction capabilities and overall performance.

Table 7. The performance of the proposed model with the attention module.

Model	Accuracy	Loss
DenseNet121	0.9601	6.0775
EfficientNetB0	0.97754	1.7494
VGG16	0.9620	2.5406
ResNet-101	0.9750	1.1080
MobileNetV1	0.9895	0.1075

Figure 17 depicts the accuracy and loss of the proposed model with the inclusion of the attention module. The accuracy (left) demonstrated a steady and consistent increase throughout the training and validation process, eventually reaching an impressive 98.95% accuracy after 40 epochs. This notable accuracy indicates the model's ability to effectively classify the data, with the aid of the attention module. Moreover, the training and validation loss (right) experienced a substantial decrease, ultimately converging to a remarkably low value of 0.1075. This loss reduction signifies that the proposed model, incorporating the attention module, effectively minimized errors during training. The results suggest that including the attention module significantly improved the model's learning capabilities, enhancing accuracy and reducing loss. These findings empirically validate the effectiveness of the attention module, highlighting its ability to capture relevant information, and improve the model's performance in complex tasks.

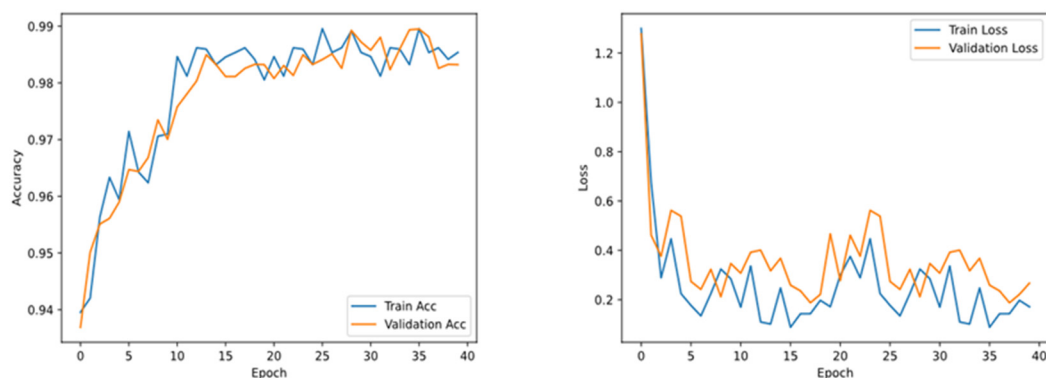
**Figure 17.** The training and validation accuracy and loss of the proposed model with the inclusion of the attention module.

Figure 18 presents the confusion matrix for the proposed model at the stage where the attention module was employed. The matrix provides detailed information about each class's correct and incorrect predictions. With the integration of the attention module, the model's performance underwent notable improvements. The overall test accuracy achieved by the proposed model using the attention module was an impressive 98.14%. This high accuracy indicates the model's ability to classify the test data across various classes accurately. Analyzing the confusion matrix allows us to identify specific classes where the model may have previously struggled, but where it showed significant improvement after incorporating the attention module. This finding emphasizes the positive impact of the attention module on enhancing the classification performance, particularly for the SC (specific class) category. The results presented in Figure 11, and the increased test accuracy, further validate the effectiveness of the attention module in improving the proposed model's performance, and its ability to capture important features and patterns within the data.



Figure 18. Confusion matrix of the proposed model with an attention module.

Table 8 presents the classification report for the proposed method incorporating the attention module, displaying the precision, recall, and F1 score. The corresponding confusion matrix, depicted in Figure 11, illustrates each class's accurate and inaccurate predictions. The test accuracy achieved was an impressive 98.96%. It is worth highlighting that the performance in SC classification witnessed a substantial improvement following the inclusion of the attention module.

Table 8. Classification report of the proposed model with an attention module.

Big 2015 Family	Precision	Recall	F1 Score	Support
Ramnit	0.99	0.99	0.99	100
Lollipop	0.98	0.97	0.97	125
Kelihos_ver3	0.99	0.97	0.99	133
Vundo	0.98	0.99	0.99	25
Simda	0.99	0.98	0.99	20
Tracur	0.99	0.99	0.99	80
Kelihos_ver1	0.98	0.99	0.99	45
Obfuscator.ACY	0.99	0.98	0.99	93
Gatak	0.99	0.99	0.99	90
Accuracy			0.99	711
Macro avg	0.99	0.99	0.99	711
Weighted avg	0.99	0.99	0.99	711

3.4.3. Comparison of the Proposed Model with State-of-the-Art Models

The current study introduces a novel attention-based deep learning model for malware recognition, which provides distinct advantages over previous machine learning and deep learning models. Prior works in this field typically relied on feature engineering techniques and specialized algorithms, or necessitated considerable human intervention for the data pre-processing and feature selection. These approaches have become inadequate, considering the rapid proliferation of new malware variants. Conversely, our proposed model employs spatial and channel attention mechanisms, in combination with a pre-trained MobileNet model, resulting in reduced trainable parameters, and a superior performance

relative to most existing deep learning models. A rigorous comparative analysis of our model's efficacy compared with previous works is presented in Table 9. Our model represents a highly sophisticated and efficient approach to addressing the intricate challenges of malware recognition.

Table 9. Comparison of the proposed model with state-of-the-art models.

Architecture	F1 Score	Accuracy	Year and Reference	Dataset
GoogleNET	...	84.0%	(2018) [24]	Maling
ResNET (A.V.G.)	...	86.01%		
CNN-SVM	...	77.22%	(2019) [29]	Maling
GRU-SVM	095%	84.92%		
MP-SVM		80.46%		
ResNet-50+ kNN	...	98.62%	(2017) [23]	Maling
Custom deep learning architecture + kNN (fast.ai)	...	94.80%	(2019) [25]	Big 2015
SVM with T + C + L features	0.85%	95.23%	(2018) [28]	Maling
kNN with T + C + L features	0.79%	96.23%		
CNN-2 layer-LSTM (7:3)	0.95%	96.4%	(2019) [30]	Maling
CNN-2 layer-LSTM (9:1)	0.95%	96.8%		
Multi-objective learning	0.95%	96.86%	(2019) [36]	Maling
Capsule network model CapsNet	...	96.58%	(2021) [46]	Maling
Kernel-based ELM with statistical texture features	0.96%	94.25%	(2020) [45]	Maling
Artificial neural network		90.80%	(2020) [51]	Big 2015
Spatial attention with VGG16	0.97%	97.62%	(2021) [2]	Maling
convolutional neural network (CNN)	0.95%	96%	2023 [48]	Maling
Convolutional neural networks		97.4%	2023 [49]	Maling
Convolutional recurrence + BiLSTM	98.36%	98.36%	2022 [50]	Big 2015
Channel attention and spatial Attention with	0.99%	98.14%	Our study	Maling
MobileNet	0.99%	98.95%	Our study	Big 2015

3.5. Ablation Study

To provide a comprehensive understanding of the effectiveness of the proposed model, we mainly performed ablation experiments, to independently evaluate the performance of each part of the proposed model as an effect of the usage of MobileNetV1, MoibleNetV1 with spatial attention, and MobileNet with spatial and channel attention, as shown in Table 10.

Table 10. Comparison of the ablation study.

Model	Dataset		Accuracy	
MobileNetV1	Maling	Big 2015	95%	95.60%
MobileNetV1 + spatial attention	Maling	Big 2015	96.5%	98.80%
The proposed model (dual attention)	Maling	Big 2015	98.14%	98.95%

3.5.1. MobileNet

Firstly, we implemented MobileNetV1 as the backbone for feature extraction. We trained the model for different epochs, ranging from 10 to 40, to track down performance. The model was overfitting on the high epochs; on the other hand, the model was biased on the lower epochs.

Additionally, we added different fully connected layers. Similarly, with a higher number of fully connected layers, the model became overfitted, and the model could not learn well on fewer fully connected layers. We adjusted the hyperparameters of the model, but the model could not improve the accuracy. Through empirical experimentation, it was

observed that the model faced difficulties in effectively learning from both datasets, and did not achieve the highest accuracy. This observation is supported by the findings in Table 1 (Maling dataset) and Table 5 (Big 2015 dataset). Despite our efforts, the model's performance fell short of the desired level on both datasets, highlighting the challenges encountered in training the model and achieving optimal accuracy. The result was found on the 40 epochs; when the epoch was increased, the model was becoming overfitted, and when the epoch was decreased, the model was not learning non-linear relationships, and showed poor generalization.

3.5.2. MobileNet with Spatial Attention

Secondly, we conducted experiments on MobileNetV1 with a spatial attention module. The accuracy was significantly improved by employing the spatial attention module. The model with the attention module extracted more relevant features and, as a result, the accuracy was improved. However, to figure out the best performance of the model, we conducted experiments on the model with the attention module, to different fully connected layers and epochs. Empirically, we found that the model with the attention module increased the accuracy, and showed good generalization. However, after a range, the model was overfitted, improving the epochs and fully connected layers.

3.5.3. MobileNet with Spatial Attention and Channel Attention

The classification model performs well if the feature extractor extracts granular or high-level features. If high-level features are fed to the classifier model, then the model will perform well, and classify more accurately. We employed MobileNet with spatial and channel attention modules, to extract essential features. The result is illustrated in Table 3 for the Maling dataset, and Table 7 for the Big 2015 dataset. There was a drastic improvement in the accuracy, and the model generalization was significantly improved. We performed experiments with varying fully connected layers and epochs, to determine the best accuracy. Through empirical evaluation, our model achieved the outstanding accuracy rate of 98.14% on the Maling dataset, and 98.95% on the Big 2015 dataset. These remarkable accuracy scores demonstrate the effectiveness and reliability of our proposed model in accurately classifying malware in both datasets.

4. Conclusions

The article proposed a novel framework for malware detection through deep learning. First, we performed data augmentation to balance the dataset, which reduced overfitting, and increased the model generalization. Next, we used transfer learning, wherein we utilized the pre-trained model MobileNetV1 as a feature extractor. Afterward, to improve the proposed model's effectiveness and accuracy, we introduced a dual-attention module, spatial attention, followed by channel attention. As a result, the framework performed very well in the detection of malware classifications. The experimental results were obtained by applying the proposed model to the Maling and Big 2015 datasets. The Maling dataset comprises 9432 grayscale images representing 25 different malware families, while the Big 2015 dataset comprises 10,868 malware images from nine other classes. On the Maling dataset, the model achieved an accuracy of 98.14%. Additionally, on the Big 2015 dataset, the model achieved the even higher accuracy of 98.95%. These results demonstrate the effectiveness of the proposed model in accurately classifying malware, while highlighting its efficient speed of operation. In this study, we conducted experiments to determine the importance of the attention module in malware detection. In future work, we would like to experiment with different attention modules, to check the most efficient and robust attention module for improvements in performance. Additionally, the transformation of grayscale images into color images is a research gap that would be a good subject for future research.

Author Contributions: Conceptualization, M.I. and A.M.A.; methodology, R.A.; software, S.H. and M.I.; validation, A.M.A., A.A. and S.H.; formal analysis, M.I.; investigation, A.A.; resources, A.M.A. and M.I.; data curation, A.A.; writing—original draft preparation, R.A.; writing—review and editing, A.M.A., M.I., S.H., R.A. and A.A.; visualization, M.I. and S.H.; supervision, M.I.; project administration, A.M.A.; funding acquisition, A.M.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Not applicable.

Acknowledgments: The researchers would like to thank the Deanship of Scientific Research, Qassim University for funding publication of this project.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Rieck, K.; Trinius, P.; Willems, C.; Holz, T. Automatic analysis of malware behavior using machine learning. *J. Comput. Secur.* **2011**, *19*, 639–668. [\[CrossRef\]](#)
2. Awan, M.J.; Masood, O.A.; Mohammed, M.A.; Yasin, A.; Zain, A.M.; Damaševičius, R.; Abdulkareem, K.H. Image-based malware classification using VGG19 network and spatial convolutional attention. *Electronics* **2021**, *10*, 2444. [\[CrossRef\]](#)
3. Kharraz, A.; Robertson, W.; Balzarotti, D.; Bilge, L.; Kirda, E. Cutting the gordian knot: A look under the hood of ransomware attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment, Proceedings of the 12th International Conference, DIMVA 2015, Milan, Italy, 9–10 July 2015*; Springer: Cham, Switzerland, 2015; pp. 3–24.
4. Kshetri, N. 1 Blockchain's roles in meeting key supply chain management objectives. *Int. J. Inf. Manag.* **2018**, *39*, 80–89. [\[CrossRef\]](#)
5. Borgia, E. The Internet of Things vision: Key features, applications and open issues. *Comput. Commun.* **2014**, *54*, 1–31. [\[CrossRef\]](#)
6. Gubbi, J.; Buyya, R.; Marusic, S.; Palaniswami, M. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Gener. Comput. Syst.* **2013**, *29*, 1645–1660. [\[CrossRef\]](#)
7. Mohammed, M.A.; Ibrahim, D.A.; Salman, A.O. Adaptive intelligent learning approach based on visual anti-spam email model for multi-natural language. *J. Intell. Syst.* **2021**, *30*, 774–792. [\[CrossRef\]](#)
8. Azeez, N.A.; Odufuwa, O.E.; Misra, S.; Oluranti, J.; Damaševičius, R. Windows PE malware detection using ensemble learning. *Informatics* **2021**, *8*, 10. [\[CrossRef\]](#)
9. Khalaf, B.A.; Mostafa, S.A.; Mustapha, A.; Mohammed, M.A.; Mahmoud, M.A.; Al-Rimy, B.A.S.; Abd Razak, S.; Elhoseny, M.; Marks, A. An adaptive protection of flooding attacks model for complex network environments. *Secur. Commun. Netw.* **2021**, *2021*, 5542919. [\[CrossRef\]](#)
10. Anam, M.; Hussain, M.; Nadeem, M.W.; Javed Awan, M.; Goh, H.G.; Qadeer, S. Osteoporosis prediction for trabecular bone using machine learning: A review. *Comput. Mater. Contin. (CMC)* **2021**, *67*, 89–105. [\[CrossRef\]](#)
11. Azizan, A.H.; Mostafa, S.A.; Mustapha, A.; Foozy, C.F.M.; Wahab, M.H.A.; Mohammed, M.A.; Khalaf, B.A. A machine learning approach for improving the performance of network intrusion detection systems. *Ann. Emerg. Technol. Comput. (AETiC)* **2021**, *5*, 201–208. [\[CrossRef\]](#)
12. Gupta, M.; Jain, R.; Arora, S.; Gupta, A.; Javed Awan, M.; Chaudhary, G.; Nobanee, H. AI-enabled COVID-19 outbreak analysis and prediction: Indian states vs. union territories. *CMC-Comput. Mater. Contin.* **2021**, *67*, 933–950. [\[CrossRef\]](#)
13. Damaševičius, R.; Venčkauskas, A.; Toldinas, J.; Grigaliūnas, Š. Ensemble-based classification using neural networks and machine learning models for windows pe malware detection. *Electronics* **2021**, *10*, 485. [\[CrossRef\]](#)
14. Awan, M.J.; Yasin, A.; Nobanee, H.; Ali, A.A.; Shahzad, Z.; Nabeel, M.; Zain, A.M.; Shahzad, H.M.F. Fake news data exploration and analytics. *Electronics* **2021**, *10*, 2326. [\[CrossRef\]](#)
15. Lal, S.; Rehman, S.U.; Shah, J.H.; Meraj, T.; Rauf, H.T.; Damaševičius, R.; Mohammed, M.A.; Abdulkareem, K.H. Adversarial attack and defence through adversarial training and feature fusion for diabetic retinopathy recognition. *Sensors* **2021**, *21*, 3922. [\[CrossRef\]](#) [\[PubMed\]](#)
16. Liu, X.; Zhang, J.; Lin, Y.; Li, H. ATMPA: Attacking machine learning-based malware visualization detection methods via adversarial examples. In *Proceedings of the International Symposium on Quality of Service*, Phoenix, AZ, USA, 24–25 June 2019; pp. 1–10.
17. Alharbi, A.; Alosaimi, W.; Alyami, H.; Rauf, H.T.; Damaševičius, R. Botnet attack detection using local global best bat algorithm for industrial internet of things. *Electronics* **2021**, *10*, 1341. [\[CrossRef\]](#)
18. Mahdavi, S.; Ghorbani, A.A. Application of deep learning to cybersecurity: A survey. *Neurocomputing* **2019**, *347*, 149–176. [\[CrossRef\]](#)
19. Nagi, A.T.; Awan, M.J.; Javed, R.; Ayesha, N. A comparison of two-stage classifier algorithm with ensemble techniques on detection of diabetic retinopathy. In *Proceedings of the 2021 1st International Conference on Artificial Intelligence and Data Analytics (CAIDA)*, Riyadh, Saudi Arabia, 6–7 April 2021; pp. 212–215.
20. Abdullah, A.; Awan, M.; Shehzad, M.; Ashraf, M. Fake news classification bimodal using convolutional neural network and long short-term memory. *Int. J. Emerg. Technol. Learn.* **2020**, *11*, 209–212.

21. Mujahid, A.; Awan, M.J.; Yasin, A.; Mohammed, M.A.; Damaševičius, R.; Maskeliūnas, R.; Abdulkareem, K.H. Real-time hand gesture recognition based on deep learning YOLOv3 model. *Appl. Sci.* **2021**, *11*, 4164. [\[CrossRef\]](#)
22. Zhuang, F.; Qi, Z.; Duan, K.; Xi, D.; Zhu, Y.; Zhu, H.; Xiong, H.; He, Q. A comprehensive survey on transfer learning. *Proc. IEEE* **2020**, *109*, 43–76. [\[CrossRef\]](#)
23. Rezende, E.; Ruppert, G.; Carvalho, T.; Ramos, F.; De Geus, P. Malicious software classification using transfer learning of resnet-50 deep neural network. In Proceedings of the 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA), Cancun, Mexico, 18–21 December 2017; pp. 1011–1014.
24. Khan, R.U.; Zhang, X.; Kumar, R. Analysis of ResNet and GoogleNet models for malware detection. *J. Comput. Virol. Hacking Tech.* **2019**, *15*, 29–37. [\[CrossRef\]](#)
25. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware images: Visualization and automatic classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security, Pittsburgh, PA, USA, 20 July 2011; pp. 1–7.
26. Nasir, M.; Muhammad, K.; Bellavista, P.; Lee, M.Y.; Sajjad, M. Prioritization and alert fusion in distributed iot sensors using kademia based distributed hash tables. *IEEE Access* **2020**, *8*, 175194–175204. [\[CrossRef\]](#)
27. Yosinski, J.; Clune, J.; Bengio, Y.; Lipson, H. How transferable are features in deep neural networks? *Adv. Neural Inf. Process. Syst.* **2014**, *27*, 3320–3328.
28. Aladhadh, S.; Alsanea, M.; Aloraini, M.; Khan, T.; Habib, S.; Islam, M. An Effective Skin Cancer Classification Mechanism via Medical Vision Transformer. *Sensors* **2022**, *22*, 4008. [\[CrossRef\]](#) [\[PubMed\]](#)
29. Akarsh, S.; Poornachandran, P.; Menon, V.K.; Soman, K. A detailed investigation and analysis of deep learning architectures and visualization techniques for malware family identification. In *Cybersecurity and Secure Information Systems: Challenges and Solutions in Smart Environments*; Springer: Cham, Switzerland, 2019; pp. 241–286.
30. Akarsh, S.; Simran, K.; Poornachandran, P.; Menon, V.K.; Soman, K. Deep learning framework and visualization for malware classification. In Proceedings of the 2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS), Coimbatore, India, 15–16 March 2019; pp. 1059–1063.
31. Kumar, S. MCFT-CNN: Malware classification with fine-tune convolution neural networks using traditional and transfer learning in Internet of Things. *Future Gener. Comput. Syst.* **2021**, *125*, 334–351.
32. Vinayakumar, R.; Alazab, M.; Soman, K.; Poornachandran, P.; Venkatraman, S. Robust intelligent malware detection using deep learning. *IEEE Access* **2019**, *7*, 46717–46738. [\[CrossRef\]](#)
33. Xiao, G.; Li, J.; Chen, Y.; Li, K. MalFCS: An effective malware classification framework with automated feature extraction based on deep convolutional neural networks. *J. Parallel Distrib. Comput.* **2020**, *141*, 49–58. [\[CrossRef\]](#)
34. Cui, Z.; Xue, F.; Cai, X.; Cao, Y.; Wang, G.-G.; Chen, J. Detection of malicious code variants based on deep learning. *IEEE Trans. Ind. Inform.* **2018**, *14*, 3187–3196. [\[CrossRef\]](#)
35. Cui, Z.; Du, L.; Wang, P.; Cai, X.; Zhang, W. Malicious code detection based on CNNs and multi-objective algorithm. *J. Parallel Distrib. Comput.* **2019**, *129*, 50–58. [\[CrossRef\]](#)
36. Jain, M.; Andreopoulos, W.; Stamp, M. Cnn vs elm for image-based malware classification. *arXiv* **2021**, arXiv:2103.13820.
37. Naeem, H.; Ullah, F.; Naeem, M.R.; Khalid, S.; Vasan, D.; Jabbar, S.; Saeed, S. Malware detection in industrial internet of things based on hybrid image visualization and deep learning model. *Ad Hoc Netw.* **2020**, *105*, 102154. [\[CrossRef\]](#)
38. Venkatraman, S.; Alazab, M.; Vinayakumar, R. A hybrid deep learning image-based analysis for effective malware detection. *J. Inf. Secur. Appl.* **2019**, *47*, 377–389. [\[CrossRef\]](#)
39. Vu, D.-L.; Nguyen, T.-K.; Nguyen, T.V.; Nguyen, T.N.; Massacci, F.; Phung, P.H. A convolutional transformation network for malware classification. In Proceedings of the 2019 6th NAFOSTED conference on information and computer science (NICS), Hanoi, Vietnam, 12–13 December 2019; pp. 234–239.
40. El-Shafai, W.; Almomani, I.; AlKhayer, A. Visualized malware multi-classification framework using fine-tuned CNN-based transfer learning models. *Appl. Sci.* **2021**, *11*, 6446. [\[CrossRef\]](#)
41. Moussas, V.; Andreatos, A. Malware detection based on code visualization and two-level classification. *Information* **2021**, *12*, 118. [\[CrossRef\]](#)
42. Roseline, S.A.; Geetha, S.; Kadry, S.; Nam, Y. Intelligent vision-based malware detection and classification using deep random forest paradigm. *IEEE Access* **2020**, *8*, 206303–206324. [\[CrossRef\]](#)
43. Verma, V.; Muttou, S.K.; Singh, V. Multiclass malware classification via first-and second-order texture statistics. *Comput. Secur.* **2020**, *97*, 101895. [\[CrossRef\]](#)
44. Çayır, A.; Ünal, U.; Dağ, H. Random CapsNet forest model for imbalanced malware type classification task. *Comput. Secur.* **2021**, *102*, 102133. [\[CrossRef\]](#)
45. Woźniak, M.; Sikka, J.; Wiecek, M.; Alrashoud, M. Recurrent neural network model for IoT and networking malware threat detection. *IEEE Trans. Ind. Inform.* **2020**, *17*, 5583–5594. [\[CrossRef\]](#)
46. Nisa, M.; Shah, J.H.; Kanwal, S.; Raza, M.; Khan, M.A.; Damaševičius, R.; Blažauskas, T. Hybrid malware classification method using segmentation-based fractal texture analysis and deep convolution neural network features. *Appl. Sci.* **2020**, *10*, 4966. [\[CrossRef\]](#)
47. Hemalatha, J.; Roseline, S.A.; Geetha, S.; Kadry, S.; Damaševičius, R. An efficient densenet-based deep learning model for malware detection. *Entropy* **2021**, *23*, 344. [\[CrossRef\]](#)

48. Depuru, S.; Hari, P.; Suhaas, P.; Basha, S.R.; Girish, R.; Raju, P.K. A Machine Learning based Malware Classification Framework. In Proceedings of the 2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT), Tirunelveli, India, 23–25 January 2023; pp. 1138–1143.
49. Yaseen, S.; Aslam, M.M.; Farhan, M.; Naeem, M.R.; Raza, A. A Deep Learning-based Approach for Malware Classification using Machine Code to Image Conversion. *Tech. J.* **2023**, *28*, 36–46.
50. Mallik, A.; Khetarpal, A.; Kumar, S. ConRec: Malware classification using convolutional recurrence. *J. Comput. Virol. Hacking Tech.* **2022**, *18*, 297–313. [[CrossRef](#)]
51. Gupta, K.; Jiواني, N.; Sharif, M.H.U.; Datta, R.; Afreen, N. A Neural Network Approach For Malware Classification. In Proceedings of the 2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS), Greater Noida, India, 4–5 November 2022; pp. 681–684.
52. Hijji, M.; Yar, H.; Ullah, F.U.M.; Alwakeel, M.M.; Harrabi, R.; Aradah, F.; Cheikh, F.A.; Muhammad, K.; Sajjad, M. FADS: An Intelligent Fatigue and Age Detection System. *Mathematics* **2023**, *11*, 1174. [[CrossRef](#)]
53. Yar, H.; Hussain, T.; Khan, Z.A.; Koundal, D.; Lee, M.Y.; Baik, S.W. Vision sensor-based real-time fire detection in resource-constrained IoT environments. *Comput. Intell. Neurosci.* **2021**, *2021*, 5195508. [[CrossRef](#)] [[PubMed](#)]
54. Kolesnikov, A.; Beyer, L.; Zhai, X.; Puigcerver, J.; Yung, J.; Gelly, S.; Houlsby, N. Big transfer (bit): General visual representation learning. In *Computer Vision—ECCV 2020, Proceedings of the 16th European Conference, Glasgow, UK, 23–28 August 2020, Part V*; Springer: Cham, Switzerland, 2020; pp. 491–507.
55. Yar, H.; Abbas, N.; Sadad, T.; Iqbal, S. Lung nodule detection and classification using 2D and 3D convolution neural networks (CNNs). In *Artificial Intelligence and Internet of Things*; CRC Press: Boca Raton, FL, USA, 2021; pp. 365–386.
56. Ali, H.; Farman, H.; Yar, H.; Khan, Z.; Habib, S.; Ammar, A. Deep learning-based election results prediction using Twitter activity. *Soft Comput.* **2021**, *26*, 7535–7543. [[CrossRef](#)]
57. Yar, H.; Khan, Z.A.; Ullah, F.U.M.; Ullah, W.; Baik, S.W. A modified YOLOv5 architecture for efficient fire detection in smart cities. *Expert Syst. Appl.* **2023**, *231*, 120465. [[CrossRef](#)]
58. Isensee, F.; Jaeger, P.F.; Kohl, S.A.; Petersen, J.; Maier-Hein, K.H. nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation. *Nat. Methods* **2021**, *18*, 203–211. [[CrossRef](#)]
59. Paymode, A.S.; Malode, V.B. Transfer learning for multi-crop leaf disease image classification using convolutional neural network VGG. *Artif. Intell. Agric.* **2022**, *6*, 23–33. [[CrossRef](#)]
60. Majeed, A.; Alnajim, A.M.; Waseem, A.; Khaliq, A.; Naveed, A.; Habib, S.; Islam, M.; Khan, S. Deep Learning-Based Symptomizing Cyber Threats Using Adaptive 5G Shared Slice Security Approaches. *Future Internet* **2023**, *15*, 193. [[CrossRef](#)]
61. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv* **2017**, arXiv:1704.04861.
62. Shaik, N.S.; Cherukuri, T.K. Multi-level attention network: Application to brain tumor classification. *Signal Image Video Process.* **2022**, *16*, 817–824. [[CrossRef](#)]
63. Yar, H.; Hussain, T.; Agarwal, M.; Khan, Z.A.; Gupta, S.K.; Baik, S.W. Optimized Dual Fire Attention Network and Medium-Scale Fire Classification Benchmark. *IEEE Trans. Image Process.* **2022**, *31*, 6331–6343. [[CrossRef](#)] [[PubMed](#)]
64. Zhao, L.; Liu, J.; Peters, S.; Li, J.; Oliver, S.; Mueller, N. Investigating the Impact of Using IR Bands on Early Fire Smoke Detection from Landsat Imagery with a Lightweight CNN Model. *Remote Sens.* **2022**, *14*, 3047. [[CrossRef](#)]
65. Ba, R.; Chen, C.; Yuan, J.; Song, W.; Lo, S. SmokeNet: Satellite smoke scene detection using convolutional neural network with spatial and channel-wise attention. *Remote Sens.* **2019**, *11*, 1702. [[CrossRef](#)]
66. Zeiler, M.D.; Fergus, R. Visualizing and understanding convolutional networks. In *Computer Vision—ECCV 2014, Proceedings of the 13th European Conference, Zurich, Switzerland, 6–12 September 2014, Part I*; Springer: Cham, Switzerland, 2014; pp. 818–833.
67. Sharma, S.; Sharma, S.; Athaiya, A. Activation functions in neural networks. *Towards Data Sci.* **2017**, *6*, 310–316. [[CrossRef](#)]
68. Abadi, M.; Agarwal, A.; Barham, P.; Brevdo, E.; Chen, Z.; Citro, C.; Corrado, G.S.; Davis, A.; Dean, J.; Devin, M. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv* **2016**, arXiv:1603.04467.
69. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*; O'Reilly Media, Inc.: Sebastopol, CA, USA, 2022.
70. Wang, C.; Zhao, Z.; Wang, F.; Li, Q. A novel malware detection and family classification scheme for IoT based on DEAM and DenseNet. *Secur. Commun. Netw.* **2021**, *2021*, 6658842. [[CrossRef](#)]
71. Abas, M.A.H.; Ismail, N.; Yassin, A.I.M.; Taib, M.N. VGG16 for plant image classification with transfer learning and data augmentation. *Int. J. Eng. Technol.* **2018**, *7*, 90–94. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.